

Resource scheduling — comparing simulated annealing with constraint programming

I B Crabtree

This paper compares two different approaches to optimisation problems. It attempts to determine which technique is best under which circumstance. On the one hand a solution-based approach was considered using simulated annealing, and on the other a state based approach using constraint propagation techniques and heuristic labelling. The aim is to show that where the problems have few constraints, the solution-based simulated annealer will perform well, but where the problems involve more constraints, then state-based constraint techniques are more appropriate.

1. Introduction

A casual glance at the literature would have you believe that every optimisation technique is good for solving every type of problem. However, this is a fallacy. It is often very difficult to choose between different techniques unless there is some clear metric that can be used to distinguish between them. To make matters worse, the techniques purported to be ideal are defined in such a general form as to allow the failures of the technique to be categorised as misinterpretations in the use of the technique or a failure to represent the problem correctly.

Some of the issues associated with the techniques currently offered as solutions to optimising resource allocation problems [1] are given below.

- Simulated annealing [2] — plenty of choice for error in selecting an appropriate cooling schedule or set of operations to manipulate the solution.
- Genetic algorithms [3] — what is the best size for the population, should the technique be steady state or generation based? Mutation rates, crossover operators, what kind of representation - so much choice and so little advice!
- Tabu search [4, 5] — issues to consider are length of the tabu list, criteria for checking against the elements in the list (i.e. what counts as similar), aspiration criteria, etc.
- Constraint programming languages [6] — representation is a big issue. If one applies the wrong representation then the whole problem is difficult to solve. Labelling the variables is an art rather than a

science. Should one select the most constrained variables first, or the ones with the smallest domain? How about variables with the largest range?

- Operations research [7] — the representation is again the sticking point. In addition, prior knowledge and experience seems crucial to make any headway, which restricts the approach to a limited elite.
- Heuristic programs — one has to be careful to employ the right heuristics to get any benefits from this approach. Finding these could prove difficult as a problem may have special characteristics. In addition, any change to the problem requires revising the heuristics which are expensive to modify.

While the above is something of a simplification, it does point to the current state of the art. It is still very much of an art, and the time is right for it to become a science. What is required is a standard approach and domain of application for each technique. This paper describes an experimental comparison of two techniques on a range of problems to determine if one should be preferred.

2. State space versus solution space

A comparison is made between simulated annealing (SA) and constraint logic programming (CLP). Simulated annealing is one of a number of solution-based approaches, generally referred to as the stochastic search techniques. An initial (usually random) solution to the problem is generated and a search for improvement is achieved by making changes to this initial solution. The way that these changes are made and the criteria for adopting a move forward in the

solution space differentiates the particular solution-based approaches.[1]

The constraint logic programming approach builds a solution incrementally and records places where choices have been made in the development of the solution. Solutions are improved by adding the constraint that further solutions must be better, and either re-starting the search, or moving back to certain decision points and exploring alternatives. The challenge in constraint programming, and any other state-based method, is to expand the state space to generate good solutions early in the search, the cost of which can then be used as a basis for constraining the search further. Lever et al [8] give an account of the current state of the art in CLP.

Simulated annealing explores the solution space, whereas constraint programming explores the state space. Figure 1 depicts how these two approaches differ.

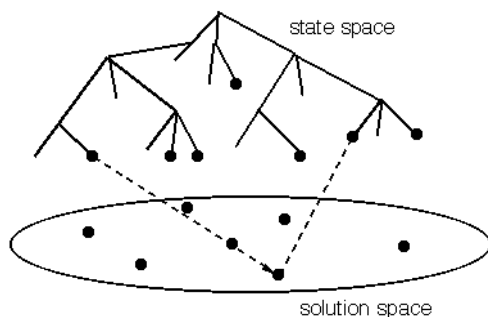


Fig 1 State space compared with solution space.

Each branch in the state space is the choice of a variable to be used and subsequently the selection of an appropriate value for that variable. As a choice is made for the value of a variable, constraints on that variable are propagated to reduce the subsequent search space and in some cases to eliminate broad areas. Filled circles represent solutions to the problem at the leaves of the state tree. In the solution-based approach an operator makes a local transformation of the solution to move about the solution space.

Because of the radical difference in the way solutions are built, solutions that are close in the solution space, may be far away from each other in the state space. This is emphasised by the dotted lines in the diagram.

The choice of simulated annealing as the solutionbased approach for the experiments described in this paper over other solution-based techniques is based on results for other resource scheduling of problems [9] where SA was the dominant stochastic technique over a range of problems. Other solution-based techniques could have been used. Indeed, genetic algorithms (GA) would be a good alternative candidate due to their radically different approach to exploring the solution space; however, GAs appear to perform well in areas where the solution space is characterised by small 'oases' of good solutions separated

by a 'desert' of poor ones. CLP has been chosen as the state space approach as it provides a powerful mechanism for automatic constraint handling together with natural representation features.

3. The problem

The problem being addressed is a general resource allocation problem that could manifest itself in a whole range of applications from co-ordination tasks, such as meeting scheduling, videoconferencing booking, timetabling and capacity planning. A clearer indication of the mapping to individual resource allocation problems is given towards the end of this section, after the details of the generic problem have been defined.

In general a resource allocation problem will specify a number of tasks and the problem will be to maximise the number of tasks achieved given a set of constraints on the task. These constraints may involve the use of limited resources or ordering constraints on the task. Flexibility in allocation is allowed by giving the task a time window in which it can be achieved to work around resource bottlenecks. Other constraints may include dependencies between tasks. Some tasks may be necessary prerequisites for other tasks to be started. The remainder of this section looks in more detail on the impact of these various constraints in the generation of a good solution.

3.1 Resource needs

Resources can be divided into two types, consumable and reusable resources. Fuel or basic materials are good examples of consumable resources, while people, tools or machines are examples of reusable resources. The problems under study make use of the second category of resource, i.e. reusable resources.

If a collection of tasks involves the use of single, or very few reusable resources from the pool of available resource, it should in principle be relatively simple to allocate them (ignoring the other factors). However, if the tasks involve many different resources as might be found in co-ordination of meetings the tasks would be difficult to schedule, because there are relatively few places where the task can be allocated. By increasing the resource needs of a task, the problem moves from a position where the tasks are easy-to-allocate with many possible alternatives, to one where they are easy-to-allocate (or not) because there are very few alternatives.

3.2 Time windows

The time window of a task defines the earliest and latest start time of the task. If the tasks are not limited to any particular time window there will be many potential places to allocate the task. Whereas if there are no time windows, and simply time points specified, the problem becomes very easy to allocate since there is little choice. Again, this

moves from a very under-constrained problem with potentially very many solutions, to a very constrained problem where there is little choice and it should therefore be very simple to determine the tasks to be achieved.

3.3 Precedence constraints

Precedence constraints define a partial ordering of tasks. With no constraints, there is total flexibility in the task allocation; however, with many ordering constraints on the allocation there will be very little choice for the allocation of the task. Yet, due to the ordering, there is less flexibility in the ordering of the tasks within their respective time windows.

There are three dimensions of flexibility in the task allocation problem which range from many through to a few solutions. Intuitively the solution-based approach might be expected to work well where there are many valid solutions, and the constraint-based approach to work well when there are very few solutions. If this is the case (and, in general, it is), the question being addressed is to determine where there is a crossover from one technique to the other, and, more importantly, what is the transition like.

4. The experiments

The set of experiments described in this paper is concerned with the allocation of 100 tasks. There are 20 different resource types, and five units of each resource. Each of the tasks takes one unit of time and makes use of a subset of the different types of resource that are available - 64 different varieties of resource allocation problem were generated by taking every combination of:

- resource needs from 10%, 30%, 60% and 80% of the available total,
- time windows varying from 1, 3, 7 and 10 units of time,
- precedence constraints from 10%, 30%, 60-90% of the tasks being dependent on others.

This gives a set of problems ranging from the underconstrained (10% of the resources, largest time windows and 10% precedence constraints), to over-constrained (80% resource types used, no time windows, and 90% of the problems with dependencies). To account for statistical variations in the generation of the problems, ten of each problem was generated, to give a total of 640 resource allocation problems. All the experiments were performed on a Sun microsystems Sparc 20 machine.

5. The simulated annealing approach

The simulated annealing approach is described in many places and so will not be repeated here. Instead the following concentrates on the effectiveness of the annealer in its exploration of the solution space. In this implementation the solution is generated by scanning a list of tasks. If a task can be allocated, it is marked as such, and the resources needed to allocate it are reduced accordingly, if the task cannot be allocated it is simply ignored. The annealer begins with a random order for the tasks in the list and each task is allocated a random start time within its time window. With this approach, there are two operators:

- a 'swap operator' that takes two tasks and swaps the order in which they are selected for allocation — tasks that are moved earlier in the list will stand a better chance of being allocated as there will be most resource available,
- a 'time change operator' that makes a random change to the time in the time window to which the task is allocated.

It is important in selecting the annealer operators that they allow (given enough time) the whole of the solution space to be sampled. Given the representation chosen for the problem the swap operator gives all tasks a chance of being allocated if they are moved to an early place in the list. The time change operator allows, for each task, the whole of the time window to be explored. With both operators there is the chance of sampling the whole of the solution space. Restricting the annealer to using just one of these operators would severely handicap its ability to sample the whole of the solution space.

It is useful to see how effective the operators are, given that the range of problems is wide. The graph in Fig 2 shows on the x axis the number of times that the time change operator was successful, and on the y axis the times that the swap operator was successful. The colour coding here goes from red (few resources per task), through purple, blue, to yellow where most resources are needed. The separation of the tasks with low resource needs (in red to the left and above) shows clearly that the swap operator is most dominant.

Colour coding with respect to the constraint dependencies paints an entirely different picture as can be seen in Fig 3. Here yellow and blue represent the problems with most dependencies between tasks. These tasks appear mainly in the top left of the graph which shows that both operators are successful for long periods of time on these kind of problems, indicating that they have to work hard 'together' to find effective solutions to problems in this region.

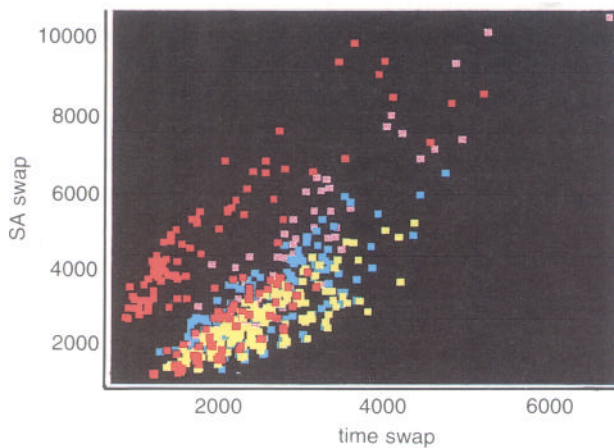


Fig 2 Effectiveness of SA operators from a resource viewpoint.

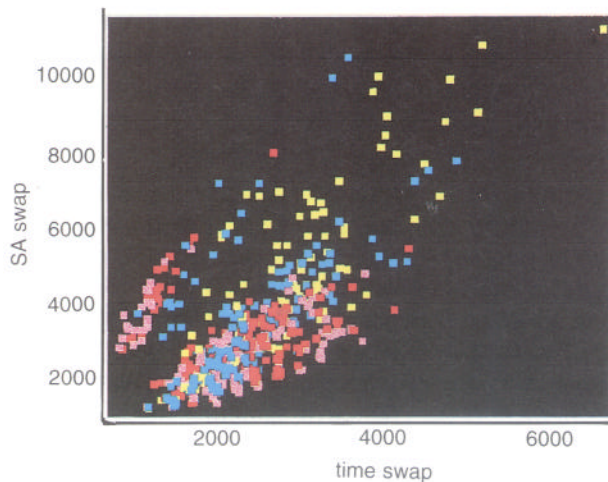


Fig 3 Effectiveness of SA operators from a task dependency viewpoint.

In setting up these experiments, initial tests were performed with the initial temperature set to zero to emulate a hillclimbing approach. In these tests the hillclimbing mode of the simulated annealer performed between 10—30% worse than the simulated annealer with an initial temperature of 10 and cooling factor of '0.9'.

The operators used here do not use any knowledge of the dependencies between tasks to select new times for tasks. It would seem reasonable that a more effective time change operator could be developed that would examine any tasks that were dependent and make a time change to the task biased towards valid allocations, i.e. make the time of the task some random time before one of its dependants. An operator to this effect was developed and tried on a subset of the problems. Rather than making a general improvement in the problems with many dependencies it made matters worse and produced, in general, worse solutions than the purely random time change operator. What appears to be happening is that although the new operator improved the local solutions it was probably having the same effect as the hillclimber, in that it was

forcing the search towards good solutions very early on and this was negating the effect of the annealer.

6. The constraint-programming approach

The key issue with constraint programming is to be able to use the constraint paradigm effectively by modelling the problem in a manner natural to the representation structure of the constraint language. In this case the platform has a finite domain language, examples of which are CHIP, ECLiPSe, and ILog's SOLVER/ SCHEDULER.

The problem is represented in a concise manner, making use of much of the natural representation of the constraint-programming language. In this problem the following have to be represented:

- the time each task is achieved,
- the constraints on resource usage,
- the dependencies between tasks,
- the problem of tasks not being completed.

One of the issues with representing these kinds of problem is that the problem is not to find if there is an assignment of all tasks, but to find an assignment that maximises the number of tasks achieved.

The representation used in this problem is based on using '0..1' variables where 0 represents a task not being allocated and a 1 indicating it will be allocated. With this method a '0..1' variable is assigned for each time point in the time window that the task can be achieved, and further constrained by ensuring the sum of these variables is in the range '0..1'. Additionally, each of these variables links to the resource constraints for the problem to ensure that resource constraints are not violated. Figure 4 shows how the time and resource constraints are linked together.

Task T_b can be assigned in the time window t_1 to t_4 , and uses resource R_i , T_j and R_k . The top equation ensures that the task will be done only once (if at all); subsequent equations represent the resource constraints on that task and how they relate to other tasks. The shading down from T_{bt2} is to indicate that the four '0..1' variables shown in the column are in fact the same variable, but spread out in this manner to depict the various interactions.

Tasks that are constrained by other tasks are linked by a delay mechanism which comes into effect whenever a task with dependencies is to be achieved; therefore any potential violations are detected when the assignment takes place.

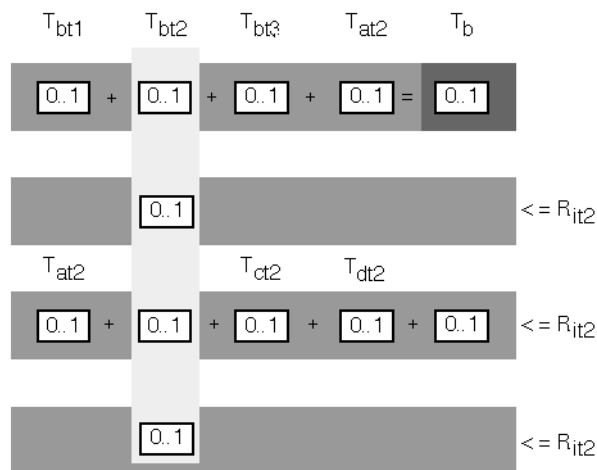


Fig 4 Model of the problem using '0...1' variables.

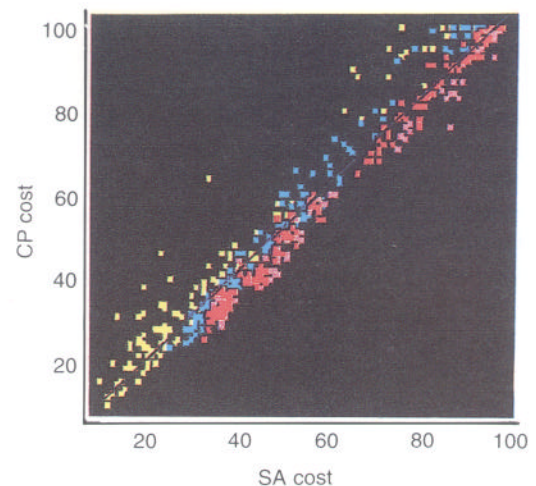
Once a suitable representation of the problem is achieved, the 'art' of constraint programming (CP) comes in the effective selection of variables and assignment of values. Selecting the variables in the wrong order can lead to poor performance in terms of the initial solution, and adversely affect the time taken to find the solution. The labelling strategy that produced the best results was based first on labelling all the tasks with dependants, and as early as possible, and leave the independent tasks until last. Current branch-and-bound methods to move towards an optimum solution do not appear to work in practice for these type of problems. In this case, early experiments were conducted where the system used branch-and-bound to move towards an optimum solution, but in most cases the method could not improve on the first solution in a reasonable amount of time. The results for the constraint programming approach are therefore the first solution found.

7. Results

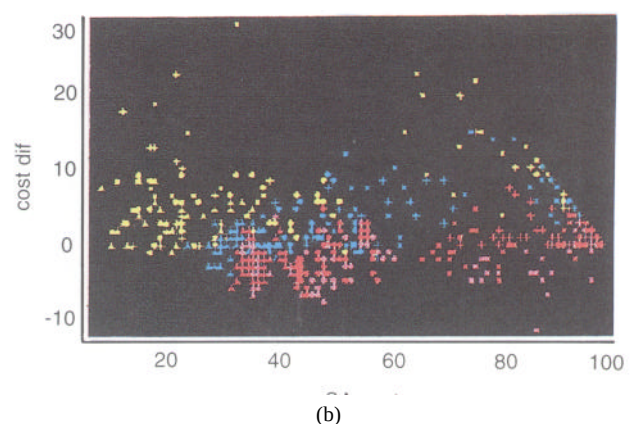
The aim of this comparative study is to examine the nature of any crossover from areas where constraint programming (CLP in this case) performs well to areas where simulated annealing also performs well. This information can then be used as a basis to select one technique over another when the problem can be determined to fit in one region or another. The following results and analysis covers overall performance and time performance.

7.1 Task allocation

Each point in the two graphs in Fig 5 plots the number of tasks allocated by simulated annealing and constraint programming. In (a) the *x* axis gives the simulated annealer result, and the *y* axis the constraint programming result; while (b) gives on the *y* axis the difference between the constraint programming approach and the simulated



(a)



(b)

Fig 5 Constraint programming versus simulated annealing.

annealing approach. Points greater than zero indicate where the constraint programming gives better results than the simulated annealer. An overall impression from the plot in (a) is that both simulated annealing and constraint programming are comparable in their solutions for this type of problem as all points are near the diagonal. However, there are regions where the constraint approach is better than the annealer and vice versa, which is made clear by the colour coding of the points on both graphs. Points coloured yellow and blue show where the constraint programming approach provides better solutions, whereas points coloured purple are solutions where the simulated annealer is better. With red points, neither technique dominates. Yellow and blue correspond to problems where there is a very high proportion of tasks that are dependent on other tasks, purple corresponds to very few dependencies between tasks. What can be seen is probably to be expected, i.e. as the dependencies (constraints) increase, then the constraint programming approach can exploit its knowledge of these constraints to the best advantage.

There appears to be a crossover in performance of the techniques where there are few dependent constraints between tasks (30% of the tasks are dependent on the

completion of other tasks). It is worth examining further the transition region from where constraint programming is dominant to where simulated annealing becomes dominant. The diagram in Fig 6 depicts that region.

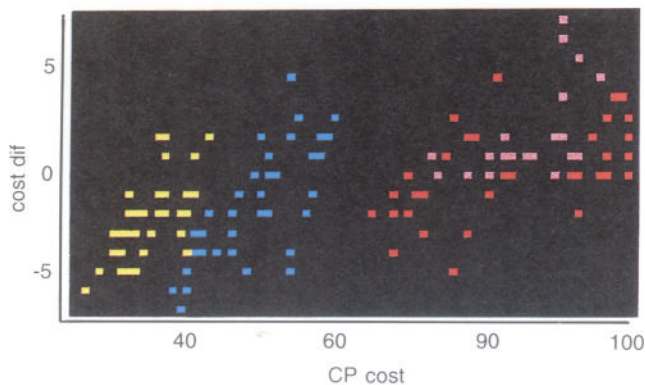


Fig 6 Crossover from constraint programming to simulated annealing.

The y axis shows the difference in cost between the constraint programming approach and the simulated annealer (cost differences greater than zero indicate an advantage to the constraint approach). The colour coding is different in this graph and indicates the resources needed to do each task. Yellow represents tasks where many different resources are used, through blue, to red to purple where tasks need the fewest resources. Clearly, where few resources are needed, then constraint programming does well; however, in the other cases, there is a spread between the constraint approach and the simulated annealer, with the annealer dominating where many resources are needed.

Here it can be concluded that, with few dependencies between tasks, it is more effective to use simulated annealing where there are many resources needed to perform each task, but, where there are few resources needed for each task, constraint programming is a more effective technique. This is an interesting result, as one would expect that, as the problem became more constrained (i.e. more resources are needed), then the constraint programming approach would be more effective, but the reverse is seen to be true. This result cannot be readily explained and is an area for further study.

Overall it is clear that the precedence constraints have the most impact on whether the constraint-programming approach dominates or the simulated-annealing approach dominates. With very few constraints, the next most dominant factor is the amount of resource used - with low resource usage the annealer is most dominant.

7.2 Performance

The previous section compared the overall effectiveness of the two techniques, and found that they were similar in overall solution, but definite demarcations could be found between the techniques. This section compares the

performance of the techniques in respect to the time taken to find their respective solutions.

This is brought out in the two graphs in Fig 7 which show for each technique the time taken to find a solution on the y axis against the cost of the solution on the x axis.

The colour coding is with respect to the size of the time windows with blue associated with time points, through purple, yellow and red representing the largest time windows. Although time windows did not have a great impact on the quality of a solution they have a major impact on the time taken to find good solutions. In addition, points that are marked with a triangle need many resources, through circles, squares to crosses which require fewest resources. Clearly from the constraint programming aspect there is a peak in the time taken to find solutions that correspond with large time windows and many resources, whereas the time taken to find good solutions with the simulated annealer has a much more distributed profile.

The diagram in Fig 8 shows a scatter plot with the x axis representing the time taken for the simulated annealer to find a solution, and the y axis representing the time for the constraint program to find a solution, the white line indicating where the time taken is the same for both techniques.

The results are very clear. In all cases, the constraint program was faster to find a solution than the simulated annealer, in some cases, up to ten times faster, but in most cases at least twice as fast as the annealer. The colour coding represents the change from very low resources needed for each task, coloured purple, through red, blue, to high resource needs in yellow.

Although this paints a particularly grim picture for the simulated annealer there are a number of points that must be borne in mind. Firstly, the criteria for stopping the search was 'no improvement in a solution after five reductions in temperature'. If the search was stopped with 'no improvement after one reduction', the time taken reduces typically by one third to one half of the original time. As improvements in the solution for a simulated annealer follow an exponential decay, the time factor could be markedly reduced.

7.3 Other issues

There are two other issues to be discussed - the scalability of each of the approaches and the memory space requirements. With the simulated-annealing approach the memory requirements are both meagre and directly proportional to the number of tasks to be scheduled, about 0.25 Mbyte for each of these experiments, whereas the constraint-programming approach used in the region of 4 Mbytes. Probably the major issue is with the scalability of

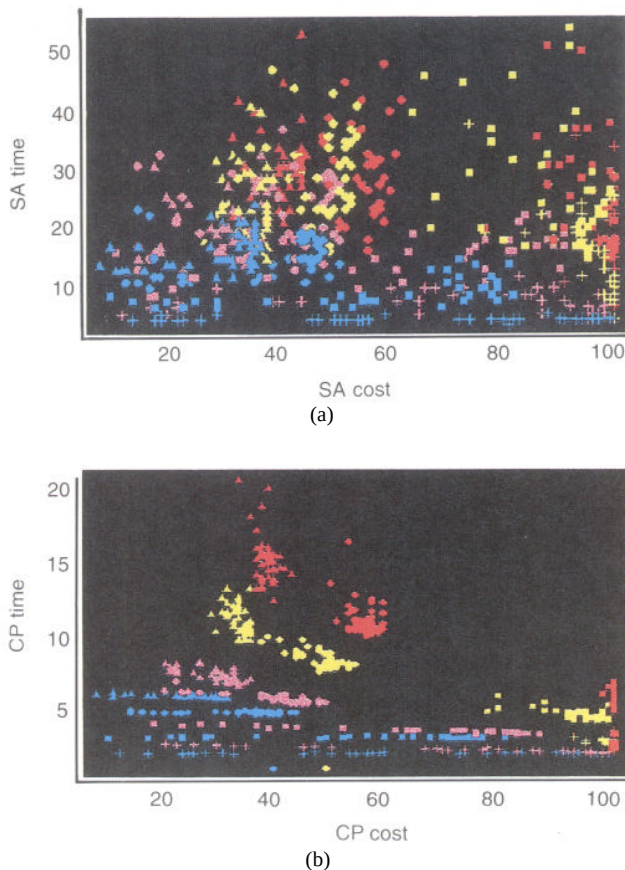


Fig 7 Time taken to find solutions for simulated annealing (a) and constraint programming (b).

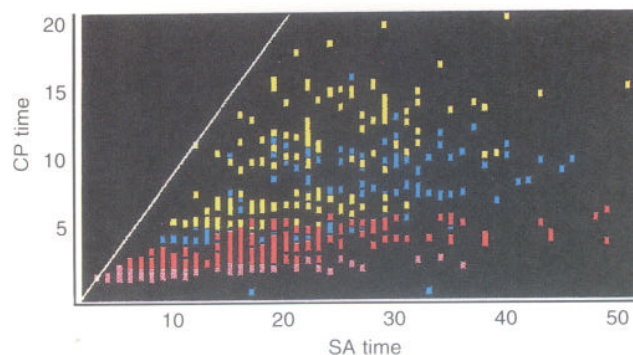


Fig 8 Time difference between constraint programming and simulated annealing.

the constraint-programming approach. Using '0..1' variables worked well in this domain because of the limited number of time windows in the problem, so the static space requirements are of the order $T \cdot W$ where T is the number of tasks and W the number of discrete time windows. There may be other representations possible that may reduce this to a constant factor, but it is likely that these results will not be of a comparable standard.

8. Conclusions

The aim of this paper has been to compare two radically different approaches to a range of resource-allocation problems in order to determine which technique would

dominate over which problems. It has been shown that there is quite a smooth transition from problems in which constraint programming (CLP in this case) is dominant to the ones in which simulated annealing is dominant, and that, surprisingly, the most influential factor is the number of precedence constraints, with the size of time windows having little effect in differentiating between constraint programming and simulated annealing.

Perhaps these results show what may be natural to expect, in that there will always be a trade-off between different techniques — clearly there is a trade-off between execution time and memory space requirements. However, it is important that experimental work is carried out to determine where one technique is preferable to another. This paper has indicated the range of issues that need to be addressed in comparing different resource-scheduling techniques. It is hoped these results will prompt other controlled comparative studies to be undertaken, thereby allowing potential users of the technology to make more informed decisions.

References

- 1 Tsang E: 'Scheduling techniques - a comparative study', *BT Technol J*, **13**, No 1, pp 16—28 (January 1995).
- 2 Kirkpatrick S et al: 'Optimisation by simulated annealing', *Science*, **200**, pp 671—680 (1983).
- 3 Holland J H: 'Adaptation in natural and artificial systems', University of Michigan Press, Ann Arbor, MI (1975).
- 4 Glover F: 'Tabu search — part I', *ORSA Journal on Computing*, **1**, No 3, pp 190—206 (1989).
- 5 Glover F: 'Tabu search — part II', *ORSA Journal on Computing*, **2**, No 1, pp 4—32 (1990).
- 6 Van Hentenryck P et al: 'Constraint satisfaction using constraint logic programming', *Artificial Intelligence Journal*, **58**, North Holland (1992).
- 7 Buchanan J T: 'Mathematical programming and resource management', *BT Technol J*, **13**, No 1, pp 29—37 (January 1995).
- 8 Lever J et al: 'Constraint logic programming for scheduling and planning', *BT Technol J*, **13**, No 1, pp 73—80 (January 1995).
- 9 Brind C et al: 'Stochastic techniques for resource management', *BT Technol J*, **13**, No 1, pp 55—63 (January 1995).



Barry Crabtree joined BT in 1980 after completing a Physics degree at Bath University. After a number of years in software development he moved into the field of artificial intelligence (AI) and helped begin BT's systems in workforce management. He is now leading a group, concentrating on research work in distributed AI, optimisation problems and adaptive systems. He has also gained a PhD in the field of logic programming and analogical reasoning.